Сборка, развёртывание и запуск

Пошаговая инструкция: локальный запуск для разработки и проверки, сборка образов, развёртывание на VPS в РФ с HTTPS через Caddy, переменные окружения, требования к ресурсам, обслуживание. Команды сверены с `docker-compose.yml`, `backend/Dockerfile`, `Makefile`, `.env.example` и скриптами `backend/scripts`. Файлы раздела 5 (`docker-compose.prod.yml`, `deploy/Caddyfile`) лежат в репозитории: compose проверен `docker compose config`, Caddyfile — `caddy validate`; локальная схема (п. 2.3) собрана и проверена с нуля в отдельном проекте compose, замеры — в п. 2.5.

Содержание: 1. Требования · 2. Локальный запуск · 3. Переменные окружения · 4. Сборка образов · 5. VPS в РФ, HTTPS через Caddy · 6. Обслуживание · 7. Диагностика · 8. В разработке

1. Требования и ресурсы

Что	Требование
ПО на машине разработки	Docker Engine 24+ с Compose v2.17+ (в `docker-compose.yml` используется `additional_contexts`), uv 0.11+, Node.js 22 LTS и npm (фронтенд, а также ключи JavaScript-заданий при генерации пула), GNU make (на Windows — команды PowerShell из п. 2.2)
Python	3.11 и выше для локального запуска (`requires-python`); в образе — 3.12
Порты на хосте (локально)	80 (интерфейс, `VZ_WEB_PORT`), 5432 (PostgreSQL), 8000 (бэкенд), 8001 (`fsp-mock`), 8025 и 1025 (Mailpit), 8080 (Keycloak, профиль `fsp`), 5173 (Vite) — переопределяются `VZ_*_PORT`
Память под ограничения контейнеров	Локальный compose: `db` 1 ГБ + `backend` 2 ГБ + `pool` 1 ГБ (только на время генерации пула) + `web` 128 МБ + `fsp-mock` 256 МБ + `mailpit` 128 МБ; в покое около 0,5 ГБ, под нагрузкой 100 пользователей около 2 ГБ (п. 2.5). С Keycloak (+768 МБ). Прод-шаблон: `db` 1 ГБ, `backend` 2 ГБ, `web` 128 МБ, `caddy` 128 МБ
Рекомендуемый VPS	4 vCPU, 8 ГБ ОЗУ для 4 процессов бэкенда (стенд жюри ~8 CPU / 32–64 ГБ — с запасом; при 2 vCPU и 4 ГБ — `VZ_WORKERS=1` – `2`). Исходно: 2 vCPU, 4 ГБ ОЗУ (сборка образов и пик Argon2: каждое хеширование занимает 64 МБ, одновременно не больше 4), 20–40 ГБ SSD. С собственным Keycloak — 6–8 ГБ ОЗУ. Это оценка по лимитам контейнеров; нагрузочное измерение на VPS не проводилось
Диск	Образы порядка 1–2 ГБ (оценка); данные PostgreSQL: пул клонов (111 семейств × 200 = около 22 тысяч экземпляров) и демо-мир — десятки мегабайт (оценка)
Домен и сеть (прод)	Доменное имя с A-записью на VPS; открытые порты 80 и 443 (выпуск сертификата Caddy и HTTPS); исходящий доступ к SMTP-серверу
Размещение	VPS с центром обработки данных **в РФ** (требование локализации ПДн, см. SECURITY.md)

2. Локальный запуск

2.1. Разработка: бэкенд на хосте (с `make`)

```
git clone <репозиторий> vzaimno && cd vzaimno
cp .env.example .env            # необязательно: значения по умолчанию подходят для локальной работы

make up-infra                   # PostgreSQL, Mailpit, fsp-mock (docker compose up -d db mailpit fsp-
mock)
make migrate                    # alembic upgrade head
make gen-pool                   # пул одноразовых клонов заданий (идемпотентно; нужен node для JS-
семейств)
make seed                       # демо-мир: ~300 кандидатов с настоящими тестами, персоны, компании
(пароли в backend/var/demo_accounts.md)
make dev                        # бэкенд с автоперезагрузкой в DEMO_MODE: http://localhost:8000/api/docs
```

```
cd frontend && npm ci && npm run dev    # интерфейс: http://localhost:5173 (Vite проксирует /api на
:8000)
```

Адрес	Что
http://localhost:8000/api/docs, `/api/redoc`, `/api/openapi.json`	Документация API (API.md)
http://localhost:8000/api/health	Готовность (БД)
http://localhost:8025	Mailpit: письма с кодами подтверждения
http://localhost:8001/docs	Демо-реестр ФСП (`fsp-mock`)
http://localhost:5173	Frontend (Vite, разработка); в контейнерной схеме интерфейс — http://localhost

Минимальная проверка без `make seed`: зарегистрироваться (`POST /api/auth/register`), прочитать код из Mailpit, подтвердить email, пройти опрос и тест (нужен пул из `make gen-pool`).

Первый запуск `make gen-pool` и `make seed` занимает несколько минут (генерация 200 клонов на семейство; `seed` дополнительно проводит настоящие тестовые сессии синтетических кандидатов). Повторные запуски идемпотентны: доливают недостающее.

Вход через ФСП ID (Keycloak). По умолчанию демо обходится без него (встроенный снимок реестра и «демо-вход ФСП»). Чтобы поднять настоящий Keycloak с realm `fsp` и 30 демо-пользователями:

```
docker compose --profile fsp up -d keycloak        # http://localhost:8080 (admin / admin); пароль демо-
спортсменов: Demo-FSP-2026
```

Для бэкенда в контейнере переменные `VZ_FSP_OIDC_*` уже заданы в `docker-compose.yml`. Для локального `make dev` задайте `VZ_FSP_OIDC_CLIENT_SECRET=vzaimno-demo-secret` (в `backend/.env` или в окружении); если вы копировали `.env.example` в `backend/.env`, очистите `VZ_FSP_OIDC_INTERNAL_URL`: адрес `keycloak:8080` виден только внутри сети docker. Подробности — FSP_INTEGRATION.md и `deploy/keycloak/README.md`.

2.2. Без `make` (Windows, PowerShell)

```
docker compose up -d db mailpit fsp-mock                          # make up-infra
cd backend
uv run alembic upgrade head                                       # make migrate
uv run python scripts/generate_pool.py                            # make gen-pool
$env:PYTHONUTF8 = "1"; uv run python scripts/seed_demo.py         # make seed
$env:VZ_DEMO_MODE = "true"; uv run uvicorn app.main:app --reload --port 8000   # make dev
uv run pytest -q                                                  # make test
uv run ruff check .                                               # make lint
uv run python -m eval.validate                                    # make validate
```

2.3. Всё в контейнерах (стенд для жюри, из чистого клона)

Нужен только Docker (Compose v2.17+); `make` необязателен (эквиваленты в комментариях).

```
git clone <репозиторий> vzaimno && cd vzaimno
docker compose up -d --build                                      # make up
curl -s http://localhost/api/health                               # ждите "pool_ready":true (PowerShell:
curl.exe); make demo ждёт сам
docker compose exec -T backend python scripts/seed_demo.py        # make demo (сам ждёт готовности пула)
docker compose exec -T backend cat var/demo_accounts.md           # make creds: логины и пароли демо-
аккаунтов
```

Сервис	Что делает	Адрес на хосте
`web`	nginx: статика Vite (SPA-fallback, gzip, `immutable` для хешированных файлов, заголовки безопасности) и прокси `/api` в `backend`	http://localhost (`VZ_WEB_PORT`)
`backend`	FastAPI; при старте `alembic upgrade head` , затем `uvicorn --workers $VZ_WORKERS` (по умолчанию 4)	http://localhost:8000
`pool`	одноразовая задача: `scripts/generate_pool.py` (пул заданий: быстрый режим — 111 семейств × 40 = 4 440 экземпляров, боевой — × 200 = 22 200; идемпотентно; несколько процессов по числу ядер), затем предрасчёт двойников (ждёт конца seed); в образе есть Node.js для JS-семейств	—
`db`	PostgreSQL 16 + pgvector (`max_connections=200`)	127.0.0.1:5432
`mailpit` , `fsp-mock`	письма с кодами, мок реестра ФСП	:8025, :8001
`keycloak`	только `--profile fsp`	:8080

Что важно:

- Локальный compose — стенд для жюри: `VZ_DEMO_MODE` по умолчанию `true` , секреты демонстрационные (в `docker-compose.prod.yml` обязательны свои).
- `pool` стартует, когда `backend` стал healthy; тест кандидата заработает, когда пул готов (`pool_ready` в `/api/health` ; `docker compose logs -f pool` ; `docker compose ps -a pool` — Exited (0) после предрасчёта двойников).
- **Быстрый режим для проверки (локальный compose по умолчанию).** `VZ_POOL_PER_FAMILY=40` в `docker-compose.yml` : 40 одноразовых клонов на семейство вместо боевых 200. Расчёт: сессия берёт из семейства не больше одного клона (≤ 20 заданий на сессию), порог долива — четверть пула (10), поэтому 40 клонов обслуживают десятки прохождений одного семейства подряд, а дальше пул доливается в фоне по ходу выдачи (так же, как в бою; seed демо доливает синхронно). Полный пул: `VZ_POOL_PER_FAMILY=200 docker compose up -d --build` (или строка в `.env`). `docker-compose.prod.yml` и `deploy/bootstrap.sh` переменную не задают — боевой пул 200. Готовность (`pool_ready`) не зависит от размера пула: не менее 80 % семейств имеют ≥ 3 свободных клона. Число процессов генерации — `VZ_POOL_WORKERS` (0 — по ядрам, не больше 8).
- Клиентский IP за nginx: nginx перезаписывает `X-Forwarded-For` адресом клиента (`VZ_REAL_IP_FROM` — кому nginx верит), бэкенд доверяет частным подсетям docker (`VZ_FORWARDED_ALLOW_IPS`).
- `restart: unless-stopped` поднимет упавший контейнер сам; healthcheck есть у всех сервисов, кроме одноразового `pool` .
- Другие порты (80 занят): `VZ_WEB_PORT=8088 VZ_PUBLIC_BASE_URL=http://localhost:8088 VZ_CORS_ORIGINS=http://localhost:8088 docker compose up -d --build` . Заняты ещё 8000/5432/8025/1025/8001 (другой стек, локальная БД) — перенесите и их: `VZ_BACKEND_PORT` , `VZ_DB_PORT` , `VZ_MAILPIT_UI_PORT` , `VZ_MAILPIT_SMTP_PORT` , `VZ_FSP_MOCK_PORT` (переменные окружения или строки в `.env` ; в PowerShell — `$env:VZ_WEB_PORT=8088`).
- Несколько процессов бэкенда и демо-режим: смещение демо-часов (`/api/demo/clock`) хранится в БД и подхватывается всеми процессами раз в секунду; счётчики лимитов запросов ведутся в памяти каждого процесса (эффективный лимит до `VZ_WORKERS` раз выше), буфер прямых POST на `/api/demo/webhook-sink` — тоже по процессу. Для строгих лимитов и показа вебхуков — `VZ_WORKERS=1` .
- `docker compose down` сохраняет данные в томе, `docker compose down -v` стирает БД и демо-мир. Сброс демо к seed без пересборки: `make reset-demo-stack` .

2.4. Тесты и валидация

```
make test        # cd backend && uv run pytest -q: более 1500 тестов; для тестов API нужен PostgreSQL
(docker compose up -d db), без него они пропускаются
make lint        # ruff check .
```

```
make validate    # симуляция и подбор → docs/VALIDATION.md, backend/eval/out/validation.json (порядка
1,5–5 минут)
```

Для тестов создаётся отдельная база `<имя>_<pid>_<rand>` на сервере из `VZ_TEST_DATABASE_URL` и удаляется после прогона.

2.5. Замеры контейнерного стенда

Окружение: Windows 11, Docker Desktop (WSL2), 20 логических ядер, 11 ГБ памяти, отдельный проект compose `vzcheck` на других портах; базовые образы (python, node, nginx, postgres) уже были в кэше Docker (на пустом кэше добавится скачивание порядка 1–2 ГБ). Нагрузка — LOAD_TEST.md.

Сборка `docker compose up -d --build` (backend, pool, web, fsp-mock и старт до `backend` healthy) — ≈ 20–60 с при тёплом кэше слоёв.

Быстрый режим (по умолчанию в локальном compose), замер 10.10.2026, чистый клон, новый том БД:

Шаг	Время
`pool` : 111 × 40 = 4 440 экземпляров в 8 процессах, 0 ошибок; `pool_ready`	≈ 70 с генерации; `pool_ready` через ≈ 105 с от `up` (последовательно те же 4 440 — ≈ 300 с)
`seed_demo.py` после `pool_ready` (300 кандидатов с настоящими тестами, персоны, 6 компаний, сценарий; пул доливается по ходу)	≈ 2 минуты
От клона до готового демо	**≈ 3 мин 50 с** (сборка с кэшем ≈ 30 с, пул ≈ 75 с, seed ≈ 2 мин)
Предрасчёт двойников (в контейнере `pool` после конца seed, до 8 потоков)	ещё ≈ 5–6 минут в фоне (всего ≈ 9,5 мин от `up`); тест и демо от него не зависят

Полный пул (`VZ_POOL_PER_FAMILY=200` , как в проде), тот же стенд:

Шаг	Время (до → после распараллеливания)
`pool` : 111 × 200 = 22 200 экземпляров, 0 ошибок; `pool_ready`	≈ 12 минут → ≈ 5,3 минуты (8 процессов; в основном тяжёлые семейства `algo.*` и JS с запуском кода)
`seed_demo.py` после `pool_ready`	≈ 70 с
От клона до готового демо	около 14–15 минут → ≈ 7 минут
Предрасчёт двойников (200 на семейство)	десятки минут → заметно короче (все семейства в общем пуле потоков), но остаётся фоновой задачей

До правок быстрого режима: пул 12 мин + seed 1 мин + сборка ≈ 1 мин = 14–15 минут, а предрасчёт двойников шёл десятки минут. Генерация детерминирована: результат зависит от семейства и seed, а не от числа процессов (тест `test_parallel_generation_equals_sequential`). На слабой машине (4 ядра) быстрый режим займёт дольше: ориентировочно 3–5 минут на пул и столько же на seed (оценка, не измерено).

Ресурсы (`docker stats`), в покое: backend ≈ 70 МБ на процесс, db ≈ 180–300 МБ, web ≈ 16 МБ, fsp-mock ≈ 37 МБ, mailpit ≈ 10–20 МБ. Под нагрузкой — LOAD_TEST.md. Образы: backend ≈ 800 МБ, web ≈ 77 МБ.

3. Переменные окружения

Бэкенд читает только переменные с префиксом `VZ_` (`app/core/settings.py`); полный список с пояснениями — `.env.example` . Продуктовые веса, пороги и квоты — не здесь, а в `config/*.yaml` . `docker compose` читает `.env` из корня репозитория (для подстановок), а сам процесс бэкенда — `.env` **из текущего каталога запуска** (для `make` это `backend/`); в контейнере значения передаёт compose.

Группа	Переменные (по умолчанию)	Заметки
Общее	`VZ_ENV` (`dev`; `test`, `prod`), `VZ_LOG_LEVEL` (`INFO`), `VZ_DEMO_MODE` (`false`), `VZ_CONFIG_DIR`	В `prod` бэкенд не стартует с dev-секретами, секретами короче 32 символов и без `VZ_COOKIE_SECURE=true`. `VZ_DEMO_MODE` включает `/api/demo/*`, автоответчик, «+1 день»
База данных	`VZ_DB_USER`, `VZ_DB_PASSWORD`, `VZ_DB_NAME` (`vzaimno`), `VZ_DB_PORT` (5432), `VZ_DATABASE_URL` (`postgresql+psycopg://vzaimno:vzaimno@localhost:5432/vzaimno`), `VZ_DB_POOL_SIZE` (10), `VZ_DB_MAX_OVERFLOW` (10), `VZ_TEST_DATABASE_URL`	Драйвер — psycopg 3 (схема `postgresql+psycopg`). В проде пароль БД поменять
Адреса	`VZ_PUBLIC_BASE_URL` (`http://localhost:5173`), `VZ_CORS_ORIGINS`, `VZ_BACKEND_PORT` (8000)	`VZ_PUBLIC_BASE_URL` — адрес фронтенда в ссылках писем и в QR сверки PDF
Секреты	`VZ_JWT_SECRET`, `VZ_APP_SECRET`, `VZ_PSEUDONYM_SECRET`	**Обязательно сменить в проде** (≥ 32 символов; `python -c "import secrets; print(secrets.token_urlsafe(48))"`). Смена `VZ_APP_SECRET` делает пул клонов нерасшифровываемым
Сессии	`VZ_ACCESS_TOKEN_TTL_S` (900), `VZ_REFRESH_TOKEN_TTL_S` (2 592 000), `VZ_REFRESH_REUSE_GRACE_S` (10), `VZ_REFRESH_COOKIE_NAME` (`vz_refresh`), `VZ_REFRESH_COOKIE_PATH` (`/api/auth`), `VZ_COOKIE_SECURE` (`false`; в проде `true`), `VZ_COOKIE_SAMESITE` (`lax`), `VZ_COOKIE_DOMAIN`	
Пароли	`VZ_PASSWORD_MIN_LENGTH` (8), `VZ_PASSWORD_MAX_LENGTH` (128), `VZ_ARGON2_TIME_COST` (3), `VZ_ARGON2_MEMORY_KIB` (65536), `VZ_ARGON2_PARALLELISM` (4), `VZ_ARGON2_MAX_CONCURRENT` (4)	Память под хеширование: `MEMORY_KIB × MAX_CONCURRENT`
Коды из писем	`VZ_EMAIL_CODE_TTL_S` (900), `VZ_EMAIL_CODE_RESEND_COOLDOWN_S` (60), `VZ_EMAIL_CODE_MAX_SENDS_PER_HOUR` (5), `VZ_EMAIL_CODE_MAX_ATTEMPTS` (5)	
Ограничение частоты	`VZ_RL_LOGIN_EMAIL` (`5/300`), `VZ_RL_LOGIN_IP` (`30/300`), `VZ_RL_VERIFY_EMAIL`, `VZ_RL_REGISTER_IP`, `VZ_RL_RESEND_IP`, `VZ_RL_REFRESH_IP`	Формат `N/секунд`
Почта	`VZ_MAIL_BACKEND` (`smtp` или `console`), `VZ_SMTP_HOST` (`localhost`), `VZ_SMTP_PORT` (1025), `VZ_SMTP_SECURITY` (`none`, `starttls`, `ssl`), `VZ_SMTP_USERNAME`, `VZ_SMTP_PASSWORD`, `VZ_SMTP_TIMEOUT_S`, `VZ_MAIL_FROM`	Локально — Mailpit. В проде — реальный SMTP по 587/465 (порт 25 на VPS часто закрыт), SPF, DKIM, DMARC на своём домене
ФСП ID и реестр	`VZ_FSP_OIDC_ISSUER`, `VZ_FSP_OIDC_INTERNAL_URL`, `VZ_FSP_OIDC_CLIENT_ID` (`vzaimno`), `VZ_FSP_OIDC_CLIENT_SECRET`, `VZ_FSP_OIDC_REDIRECT_URI`, `VZ_FSP_SCOPES`, `VZ_FSP_REGISTRY_URL`, `VZ_FSP_PROVIDER` (`auto`; `http`, `inprocess`), `VZ_FSP_REGISTRY_TOKEN`, `VZ_FSP_TOKEN_URL`, `VZ_FSP_REGISTRY_SCOPE`, `VZ_FSP_TIMEOUT_S` (5), `VZ_FSP_DEMO_LOGIN_ENABLED` (`true`), `VZ_FSP_STATE_TTL_S` (600), `VZ_FSP_RL_*`	Переход на настоящий ФСП ID — FSP_INTEGRATION.md п. 9. Keycloak: `VZ_KEYCLOAK_ADMIN`, `VZ_KEYCLOAK_ADMIN_PASSWORD`, `VZ_KEYCLOAK_PORT`, `VZ_FSP_KEYCLOAK_PUBLIC_URL`, `VZ_PUBLIC_DOMAIN`
Резюме и PDF	`VZ_RESUME_MAX_BYTES` (5 МБ), `VZ_RESUME_MAX_PAGES` (10), `VZ_RESUME_MIN_TEXT_CHARS` (40), `VZ_RESUME_RL_UPLOAD`, `VZ_PROFILE_RL_PDF`, `VZ_PROFILE_RL_PREVIEW`, `VZ_PROFILE_RL_SEARCH`, `VZ_PROFILE_RL_VERIFY`	
LLM (необязательно)	`VZ_LLM_PROVIDER` (`none`; `auto`, `yandexgpt`, `gigachat`), `VZ_LLM_MODE` (`none`, `live`, `record`, `replay`), `VZ_LLM_CASSETTE_PATH`, `VZ_LLM_TIMEOUT_S` (20), `VZ_LLM_MIN_QUOTE_RATIO` (90), `VZ_LLM_YANDEX_API_KEY`, `VZ_LLM_YANDEX_FOLDER_ID`, `VZ_LLM_GIGACHAT_AUTH_KEY` и др.	По умолчанию выключена. Включать только с реальными ключами; ПДн маскируются до вызова
За обратным прокси	`VZ_FORWARDED_ALLOW_IPS` (или `FORWARDED_ALLOW_IPS`, как у uvicorn; по умолчанию `127.0.0.1`)	**Обязательна за Caddy**: адреса или подсеть прокси (п. 5.3); `*` в prod запрещено

Группа	Переменные (по умолчанию)	Заметки
Демо на боевом стенде	`VZ_DEMO_CONTROLS_ADMIN_ONLY` (по умолчанию: `true` в `VZ_ENV=prod`), `VZ_DEMO_CONTROL_KEY`	При `VZ_DEMO_MODE=true` общие кнопки демо (часы, сброс, автоответчик, режим жюри) в prod — только админ или заголовок `X-Demo-Key`; вне prod — любая роль
Планировщик	`VZ_SCHEDULER_ENABLED` (по умолчанию — `config/scheduler.yaml`)	Фоновые задачи внутри бэкенда (п. 6); в тестах не запускается

4. Сборка образов

```
# Бэкенд: python:3.12-slim, uv 0.11.12, зависимости строго по uv.lock, процесс от непривилегированного пользователя
docker compose build backend
# вручную (config/*.yaml лежит вне контекста backend и передаётся отдельным контекстом «config»):
cd backend && docker build --build-context config=../config -t vzaimno-backend .

# Мок реестра ФСП
docker build -t vzaimno/fsp-mock services/fsp-mock

# Фронтенд: multi-stage (node 22 → nginx 1.27), статика и прокси /api внутри образа
docker compose build web
```

В образ бэкенда входят `app/`, `alembic/`, `alembic.ini`, `config/*.yaml`, `eval/` (нужен демо-модулям), `scripts/` (пул, seed) и бинарник Node.js (песочница JS-заданий); тесты не входят. Healthcheck образа — `GET /api/health/live`.

5. Развёртывание на VPS в РФ с HTTPS через Caddy

Схема: наружу открыты только 80 и 443 (Caddy); бэкенд и БД доступны лишь внутри сети compose (БД не публикуется вообще). Caddy терминирует TLS, проксирует `/api/*` в бэкенд, остальное — в контейнер `web` (nginx со статикой, SPA-fallback и кэшем), сам получает и продлевает сертификаты (ACME).

Стенд жюри одной командой (чистый Ubuntu 22.04/24.04 или Debian 12, 4 vCPU / 8 ГБ рекомендуется, открыты 80 и 443):

```
git clone <репозиторий> /opt/vzaimno && cd /opt/vzaimno && sudo bash deploy/bootstrap.sh
```

`deploy/bootstrap.sh` ставит Docker (если его нет), добавляет swap при ОЗУ меньше 6 ГБ, генерирует `.env` с секретами, поднимает `docker-compose.prod.yml`, ждёт бэкенд и пул заданий, засевает демо-мир и проверяет `https://<домен>/api/health`. Без `--domain` адрес стенда — `<ip-через-дефисы>.sslip.io` (публичный wildcard-DNS; Caddy выпускает на него сертификат Let's Encrypt, покупать домен не нужно). Без SMTP письма не уходят, а в демо-режиме регистрация подтверждается автоматически. Повторный запуск (`git pull && sudo bash deploy/bootstrap.sh`) безопасен: `.env` не перезаписывается. Параметры — `bash deploy/bootstrap.sh --help`. Ниже — те же шаги вручную.

5.1. Подготовка сервера

1. Арендовать VPS в дата-центре РФ (параметры — п. 1), установить Debian или Ubuntu LTS, обновить систему.
2. Создать DNS-запись `A` для домена (например, `vzaimno.example`) на адрес VPS; подождать распространения.
3. Установить Docker Engine и плагин Compose v2 (по инструкции дистрибутива или проекта Docker).

4. Открыть в межсетевом экране только 22 (SSH), 80 и 443. Помните: порты, опубликованные Docker, **обходят** `ufw` — поэтому в прод-схеме БД и бэкенд не публикуются вовсе.
5. Настроить доступ по SSH-ключам, отключить вход по паролю.

5.2. Код, секреты, фронтенд

```
git clone <репозиторий> /opt/vzaimno && cd /opt/vzaimno

# Секреты: три разных значения, каждое не короче 32 символов
python3 -c "import secrets; print(secrets.token_urlsafe(48))"      # выполнить трижды

cat > .env <<'EOF'
VZ_PUBLIC_DOMAIN=vzaimno.example
VZ_DB_PASSWORD=<длинный случайный пароль>
VZ_JWT_SECRET=<секрет 1>
VZ_APP_SECRET=<секрет 2>
VZ_PSEUDONYM_SECRET=<секрет 3>
VZ_SMTP_HOST=smtp.<ваш-провайдер>.ru
VZ_SMTP_PORT=587
VZ_SMTP_SECURITY=starttls
VZ_SMTP_USERNAME=<логин>
VZ_SMTP_PASSWORD=<пароль>
VZ_MAIL_FROM=Взаимно <no-reply@vzaimno.example>
# Стенд для жюри: VZ_DEMO_MODE=true, VZ_FSP_DEMO_LOGIN_ENABLED=true; боевой режим — оставить false
VZ_DEMO_MODE=false
EOF
chmod 600 .env

# Фронтенд собирается в образе web при `docker compose -f docker-compose.prod.yml up -d --build`: Node.js
на VPS не нужен
```

`.env` хранить вне git и вместе с резервной копией БД: без `VZ_APP_SECRET` зашифрованные ключи клонов не прочитать.

5.3. Файлы развёртывания

В репозитории лежат готовые файлы (compose проверен `docker compose config`, Caddyfile — `caddy validate`; запуск с настоящим доменом и сертификатом не выполнялся):

- `docker-compose.prod.yml` — боевая схема: `db`, `backend`, `pool` (одноразовая генерация пула), `web`, `caddy`; профиль `fsp` добавляет свой ФСП ID (`keycloak-db` + `keycloak`);
- `deploy/Caddyfile` — HTTPS, заголовки безопасности, проксирование `/api/*` в backend и остального в `web`;
- `deploy/docker-compose.db-loopback.yml` — временная публикация БД на `127.0.0.1` для генерации пула с хоста.

Запуск: `docker compose -f docker-compose.prod.yml up -d --build`. Что отличает схему от локального `docker-compose.yml`:

Мера	Как сделано
БД не публикуется	У `db` нет `ports`: PostgreSQL виден только соседним контейнерам. Для обслуживания — `docker compose -f docker-compose.prod.yml exec db psql -U vzaimno`; для генерации пула с хоста — временный `deploy/docker-compose.db-loopback.yml` (только `127.0.0.1`, после работы пересоздать `db` без него) или `exec` внутри контейнера
Бэкенд не публикуется	`expose: 8000`, наружу только `caddy` (80, 443, 443/udp)

Мера	Как сделано
Версии закреплены	`pgvector/pgvector:0.8.7-pg16`, `caddy:2.10.2-alpine`, `postgres:16.15-alpine`, `quay.io/keycloak/keycloak:26.0.8` — вместо плавающих `pg16` / `latest`. Обновление — осознанная правка версии в файле. Mailpit в прод-схему не входит (локальный compose закрепляет `axllent/mailpit:v1.31.4`). Для жёсткого закрепления можно дописать `@sha256:<digest>` к тегу (`docker buildx imagetools inspect <образ>`)
Keycloak не `start-dev`	В профиле `fsp` — production-режим `start` (с `--hostname`, `--proxy-headers=xforwarded`, своей PostgreSQL). `start-dev` остался только в локальном compose. Обязательные переменные профиля: `VZ_KEYCLOAK_DB_PASSWORD`, `VZ_KEYCLOAK_ADMIN`, `VZ_KEYCLOAK_ADMIN_PASSWORD`, `VZ_FSP_KEYCLOAK_PUBLIC_URL` (`https://sso.<домен>`); для доступа браузера раскомментируйте блок `{$VZ_KEYCLOAK_DOMAIN}` в `deploy/Caddyfile` (админ-консоль наружу не отдавать)
Клиентский IP за Caddy	Подсеть compose зафиксирована (`172.29.80.0/24`), бэкенд доверяет `X-Forwarded-For` только от неё (`VZ_FORWARDED_ALLOW_IPS`), Caddy перезаписывает заголовок адресом клиента, uvicorn запущен с `--no-proxy-headers` (см. ниже)
Демо-кнопки	`VZ_DEMO_CONTROL_KEY` (необязателен): в prod при `VZ_DEMO_MODE=true` часы, сброс, автоответчик и переключатель режима жюри — только роль `admin` или заголовок `X-Demo-Key`

Замечания:

- **`VZ_FORWARDED_ALLOW_IPS`** (читается и `FORWARDED_ALLOW_IPS`, как у uvicorn): список адресов и подсетей через запятую, по умолчанию `127.0.0.1`. Приложение (`app/core/netutil.py`) верит заголовку `X-Forwarded-For` только от этих соседей и читает цепочку справа налево. Caddy стоит в соседнем контейнере, поэтому без этой переменной все пользователи выглядят как один IP прокси и **делят общие лимиты** (вход, регистрация, обновление сессии); в prod значение `*` запрещено — иначе любой, кто дотянется до порта бэкенда, подставлял бы себе IP. Если в compose используется другая подсеть — поменяйте её в `networks.default.ipam` и в этой переменной одновременно.
- **`VZ_PUBLIC_BASE_URL`** должен совпадать с публичным адресом: он попадает в письма, QR и адрес возврата из ФСП ID.
- **SPA-маршруты**: `try_files … /index.html` нужен для адресов вида `/candidate/fsp/callback`.
- **Реестр ФСП**: по умолчанию `VZ_FSP_PROVIDER=inprocess` (встроенный демо-снимок, `fsp-mock` не нужен). Для боевого режима — переменные из FSP_INTEGRATION.md п. 9 (`VZ_FSP_PROVIDER=http`, адреса и учётные данные ФСП ID): добавьте их в `environment` сервиса `backend` (пустых значений не задавайте).
- Mailpit и `fsp-mock` из локального compose в прод-схему **не входят**.

Пересборка образа бэкенда (после `git pull`, смены зависимостей или кода; образ собирается заново, старый контейнер не «подхватывает» новые разделы API):

```
docker compose -f docker-compose.prod.yml build --no-cache backend     # зависимости строго по uv.lock
docker compose -f docker-compose.prod.yml up -d backend                # миграции применяются при старте
docker compose -f docker-compose.prod.yml ps                           # backend должен стать healthy
curl -s https://vzaimno.example/api/openapi.json | head -c 200         # новая схема: проверьте, что есть
нужные маршруты
```

Локально (`docker-compose.yml`): `docker compose build --no-cache backend && docker compose up -d backend`; если на `:8000` отвечает старый образ без новых разделов (например, `GET /api/bank/summary` даёт 404), причина — устаревший образ, а не код: пересоберите его, как выше.

5.4. Запуск, миграции, пул клонов, демо-мир

```
docker compose -f docker-compose.prod.yml up -d --build        # миграции применяются при старте backend
docker compose -f docker-compose.prod.yml ps                   # backend должен стать healthy
curl -s https://vzaimno.example/api/health                     # {"status":"ok","db":"ok",...}
```

Пул клонов обязателен: без него тест не стартует. Его создаёт одноразовый сервис `pool` (в образе есть `scripts/` и Node.js): он стартует сам после `backend` и занимает ≈ 5 минут на 20 логических ядрах ноутбука (8

процессов; до распараллеливания ≈ 12 минут), на 2–4 vCPU заметно дольше (оценка, не измерено; `VZ_POOL_WORKERS` по умолчанию 4 в prod-compose) (индикатор «Готовим одноразовые задания» на /jury; готовность — `"pool_ready":true` в `/api/health`) (`docker compose -f docker-compose.prod.yml logs -f pool`). Повтор или долив: `docker compose -f docker-compose.prod.yml run --rm pool` ; состояние: `... run --rm pool python scripts/generate_pool.py --stats` . `VZ_APP_SECRET` у `pool` и `backend` один (ключи шифруются им). Туннель к БД для этого больше не нужен; `deploy/docker-compose.db-loopback.yml` оставлен для обслуживания.

Демо-мир для стенда жюри (`VZ_DEMO_MODE=true` в `.env`): `seed_demo.py` отказывается работать при `VZ_ENV=prod` , поэтому его запускают с машины с туннелем с локальным `VZ_ENV=dev` и тем же `VZ_APP_SECRET` (пароли демо-аккаунтов выводятся из него); через туннель наполнение идёт медленнее, чем локально:

```
cd backend
VZ_ENV=dev uv run python scripts/seed_demo.py              # пароли — в backend/var/demo_accounts.md (в
.gitignore)
```

Боевой режим (`VZ_DEMO_MODE=false`): `seed_demo.py` не запускать, маршруты `/api/demo/*` отвечают 404.

5.5. Проверка после развёртывания

Проверка	Ожидание
`https://<домен>/`	Открывается фронтенд по HTTPS (сертификат Caddy)
`https://<домен>/api/health`	`status: ok` , `db: ok`
`https://<домен>/api/docs`	Swagger UI
Регистрация нового пользователя	Письмо с кодом приходит (реальный SMTP), `Set-Cookie: vz_refresh` с флагами `HttpOnly; Secure; Path=/api/auth`
Серия неверных входов	После 5 попыток за 300 с — 429 (проверяет, что клиентский IP определяется верно: 429 получает тот, кто ошибался, а не все)
`GET /api/fsp/availability`	Корректный `registry_label`
Тест	Опрос → `POST /api/tests/sessions` → задания выдаются (пул создан)
Журнал	`docker compose -f docker-compose.prod.yml logs backend` без ошибок

6. Обслуживание

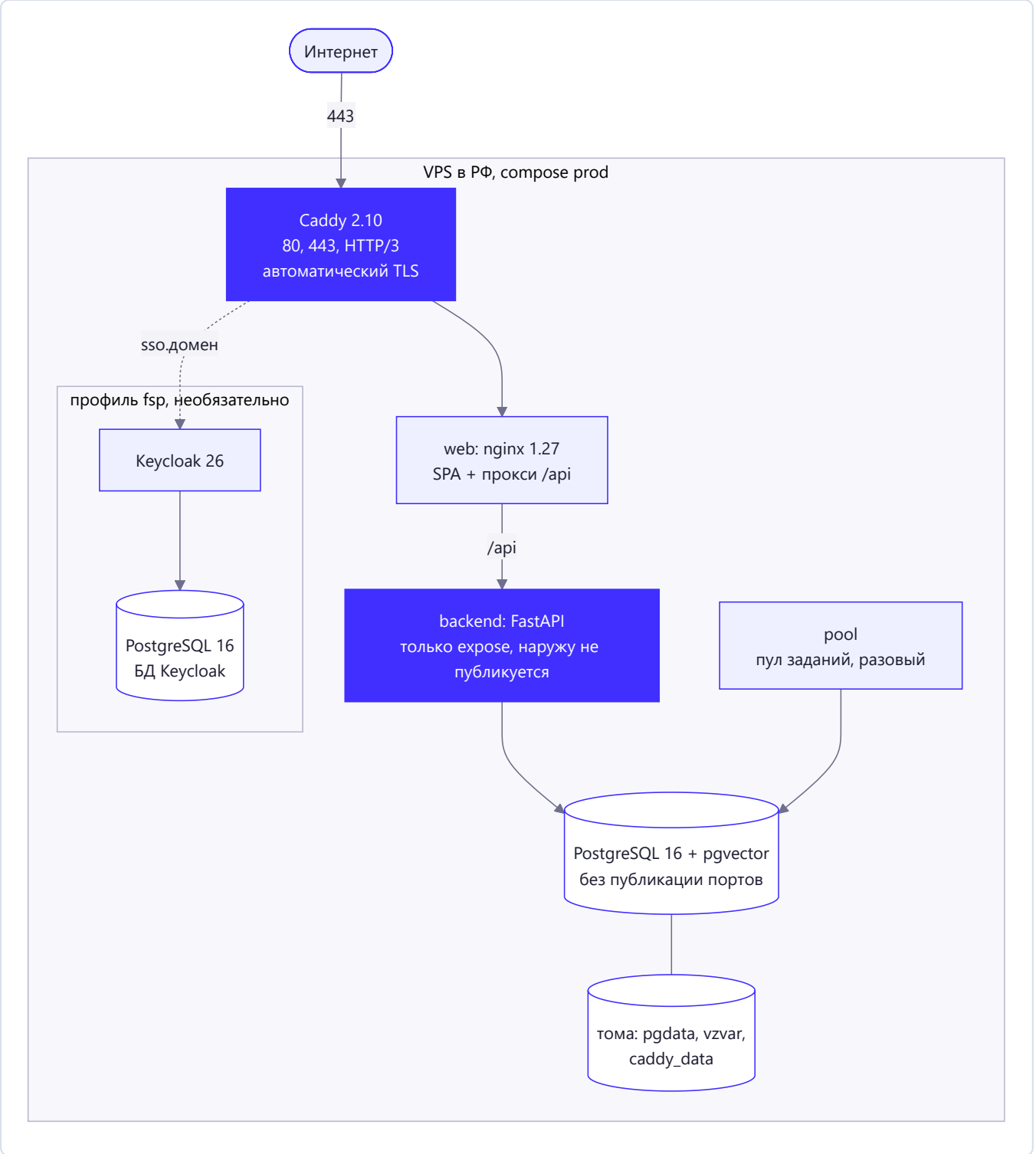
Задача	Команда
Резервная копия БД	`docker compose -f docker-compose.prod.yml exec -T db pg_dump -U vzaimno vzaimno | gzip > backup-$(date +%F).sql.gz` ; копии и `.env` хранить на отдельном носителе в РФ
Восстановление	`gunzip -c backup.sql.gz | docker compose -f docker-compose.prod.yml exec -T db psql -U vzaimno vzaimno` в пустую БД
Обновление версии	`git pull && cd frontend && npm ci && npm run build && cd .. && docker compose -f docker-compose.prod.yml up -d --build` (миграции применяются автоматически); при новых семействах — повторить `generate_pool.py`
Откат	`git checkout <прежний тег>` и тот же `up -d --build` ; миграции Alembic вниз (`alembic downgrade`) выполняются только вручную и осознанно
Долив пула	Идёт автоматически в фоне после выдачи заданий (когда свободных меньше 50); вручную — `generate_pool.py`
Калибровка сложности	Ночью — планировщик (`config/scheduler.yaml` , 00:00 UTC = 03:00 МСК); вручную — `POST /api/lab/calibrate` (роль методист)

Задача	Команда
Фоновые задачи	Встроенный планировщик (asyncio в процессе бэкенда): раз в минуту — сроки приглашений и откликов и повтор вебхуков, ночью — калибровка с CUSUM и чистка по срокам хранения. Итог суточных задач — в `audit_log` (`scheduler.calibration` , `scheduler.retention`); отключить — `VZ_SCHEDULER_ENABLED=false`
Логи	`docker compose -f docker-compose.prod.yml logs -f backend` ; уровень — `VZ_LOG_LEVEL`
Здоровье	`GET /api/health` (БД), `GET /api/health/live` (процесс); у контейнеров есть healthcheck
Смена секретов	`VZ_JWT_SECRET` — пользователи выйдут; `VZ_APP_SECRET` — пул клонов придётся пересоздать; `VZ_PSEUDONYM_SECRET` — новые псевдонимы для новых показов

7. Диагностика

Симптом	Причина и решение
Бэкенд падает при старте: «VZ_..._SECRET обязательно задать в prod» или «не короче 32 символов»	В `VZ_ENV=prod` запрещены dev-секреты; задайте три секрета в `.env`
«В prod refresh-cookie должна быть Secure»	Задайте `VZ_COOKIE_SECURE=true` (в `docker-compose.prod.yml` уже есть)
429 у всех пользователей, в логах один IP	Не задан `VZ_FORWARDED_ALLOW_IPS` за прокси (п. 5.3)
Пользователь не остаётся залогиненным	Нет HTTPS при `VZ_COOKIE_SECURE=true` , неверный `VZ_PUBLIC_DOMAIN` или `VZ_CORS_ORIGINS`
Письма не приходят	Порт 25 закрыт у провайдера: используйте 587 или 465; проверьте SPF, DKIM, DMARC; смотрите `logs backend` на «SMTP: письмо ... не отправлено»; ответ 503 `mail_unavailable`
Тест не стартует, «банк исчерпан» или нет заданий	Не создан пул: `generate_pool.py` (п. 5.4)
«Не удалось расшифровать ключ экземпляра»	Пул создан с другим `VZ_APP_SECRET` : верните прежний секрет либо удалите невыданные экземпляры и создайте пул заново
`generate_pool.py` : «node не найден»	Нужен Node.js в `PATH` для семейств Frontend
Windows: ошибки кодировки в консоли	`PYTHONUTF8=1` (`make` выставляет сам)
`additional_contexts` не поддерживается	Обновите Docker Compose до v2.17+

Схема боевого развёртывания



8. В разработке

- Мониторинг, алерты и автоматическое резервное копирование.
- Закрепление образов по digest (сейчас — по версиям; `python:3.12-slim` в Dockerfile остаётся плавающим тегом минорной линии).